

Restaurant Ordering System

A Multi-Tenant PostgreSQL Database for Small Restaurant Chains

Odiljon Asmatov

AUCA · Software Engineering · 2026



PostgreSQL 16



FastAPI



Next.js

31 Tables

18 RLS Tables

ACID Compliant

Problem & Solution

PROBLEMS



Fragmented data

Orders, inventory, loyalty live in separate systems



No tenant isolation

One branch can see another branch's data



Race conditions

Two customers grab the last portion at the same time

SOLUTION



Single PostgreSQL database

Orders + inventory + loyalty unified



Row-Level Security

Data isolation enforced at the DB level



ACID transactions

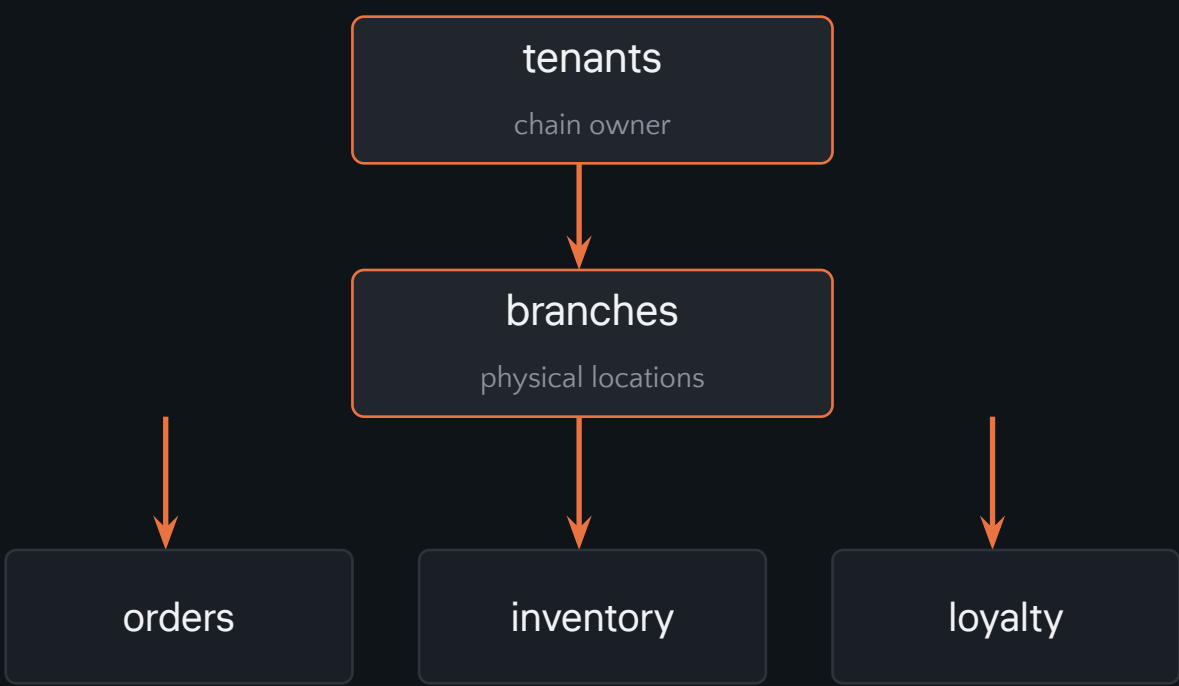
Atomic orders via stored procedures

Tech Stack and Database Structure

TECH STACK



DATA HIERARCHY



Core: 10	Inventory: 8	Loyalty: 6	Orders: 4	Audit: 3	31 tables
----------	--------------	------------	-----------	----------	-----------

What I Used from PostgreSQL



Row-Level Security

- 18 tables protected
- Auto-filtered by tenant_id
- Enforced on every query
- Defense-in-depth at DB level



Table Partitioning

- orders partitioned by month
- orders_2024_01, 2024_02, ...
- Faster date-range queries
- Simple archival of old data



Indexes

- B-Tree — search and FKs
- GIN — JSONB and arrays
- BRIN — partitions (minimal storage)

18 / 31 tables

PARTITION BY RANGE

3 index types

Plus: 3NF · Foreign Keys · CHECK constraints · Audit triggers (logged UPDATE/DELETE)

fn_place_order — the Heart of the System

PL/pgSQL

```
BEGIN;  
-- 1. Lock inventory rows  
SELECT * FROM inventory  
WHERE branch_id = $1 FOR UPDATE;  
-- 2. Check stock availability  
IF stock >= quantity THEN  
-- 3. Deduct ingredients  
UPDATE inventory SET qty = qty - $2;  
-- 4. Create order record  
INSERT INTO orders (...) VALUES (...);  
-- 5. Update loyalty points  
UPDATE loyalty_accounts SET points = ...;  
END IF;  
COMMIT;
```

ACID GUARANTEES

A Atomic

All or nothing

I Isolated

No interference

C Consistent

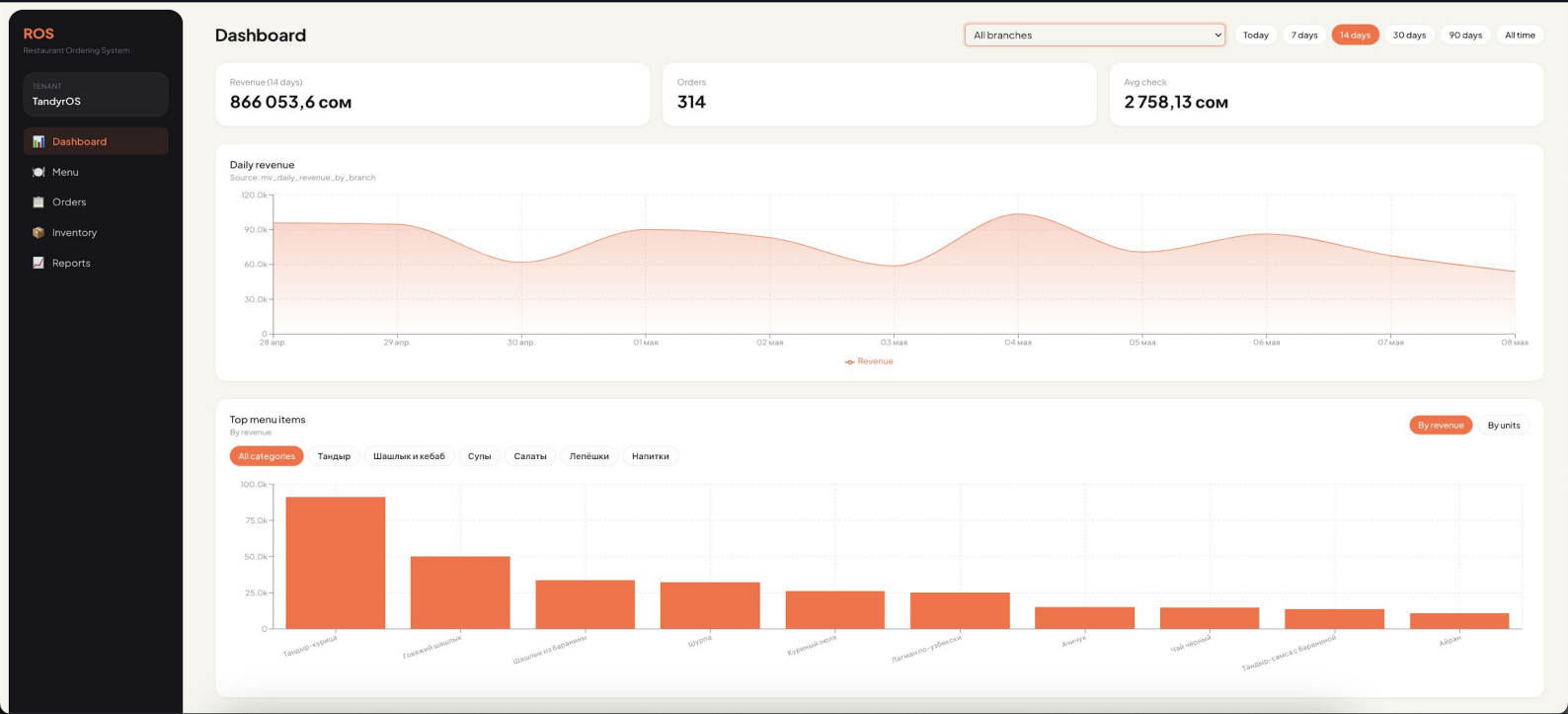
Valid state only

D Durable

Persisted forever

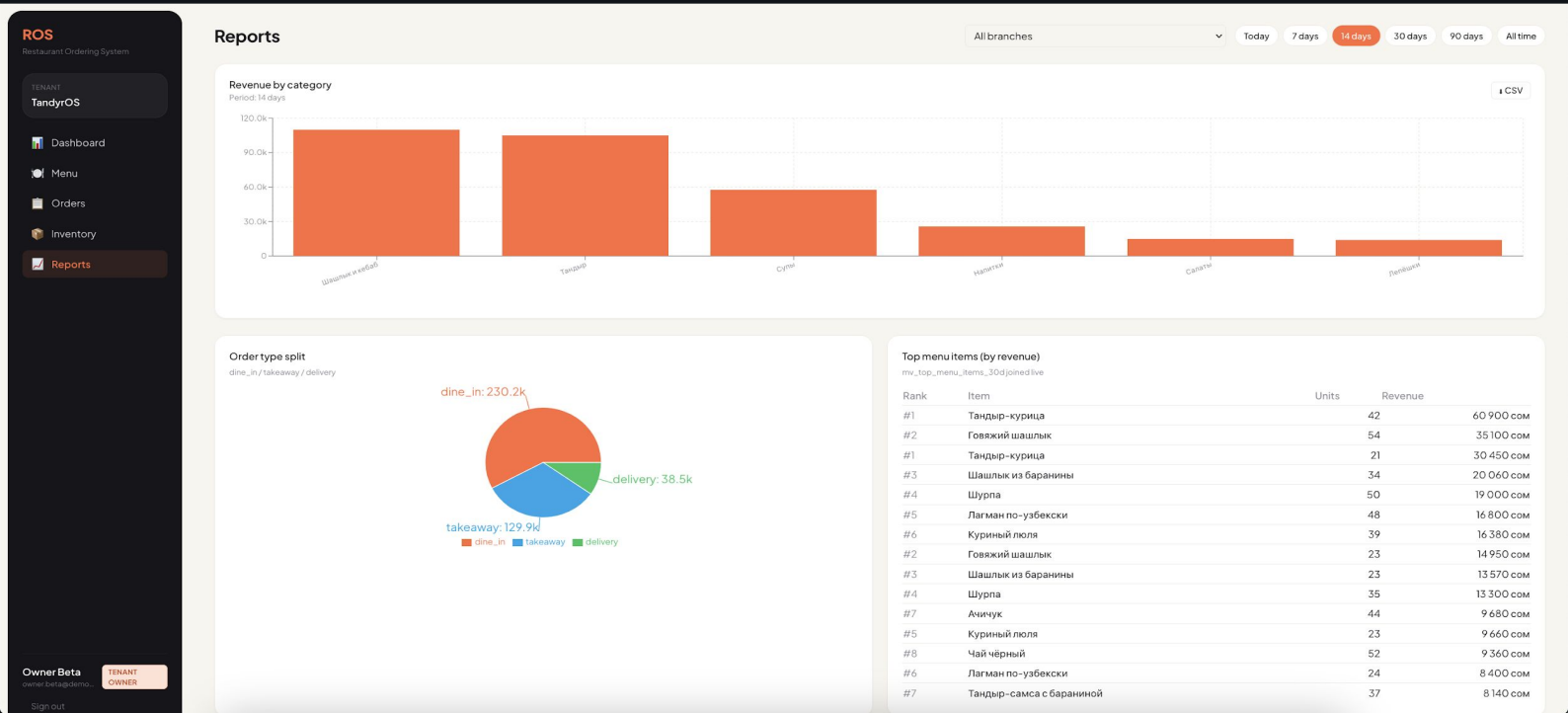
i **FOR UPDATE** lock prevents race conditions: the second order waits until the first one finishes.

Materialized Views in Action



Dashboard

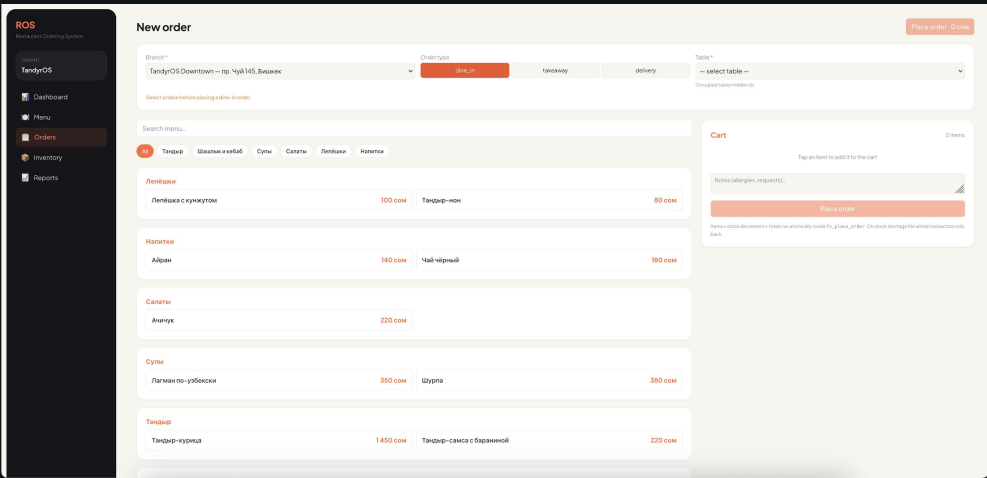
- Revenue chart → mv_daily_revenue_by_branch
- Top menu items → materialized view
- Tenant filter → RLS applied automatically



Reports

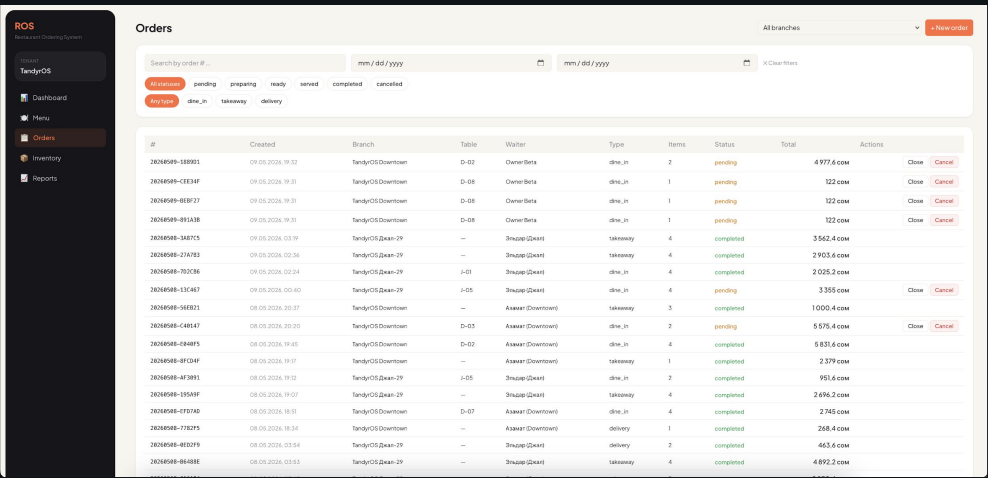
- Revenue by category → category aggregation
- Order type split → dine_in / takeaway / delivery
- Top items → mv_top_menu_items_30d

Orders Flow and Inventory Tracking



New Order

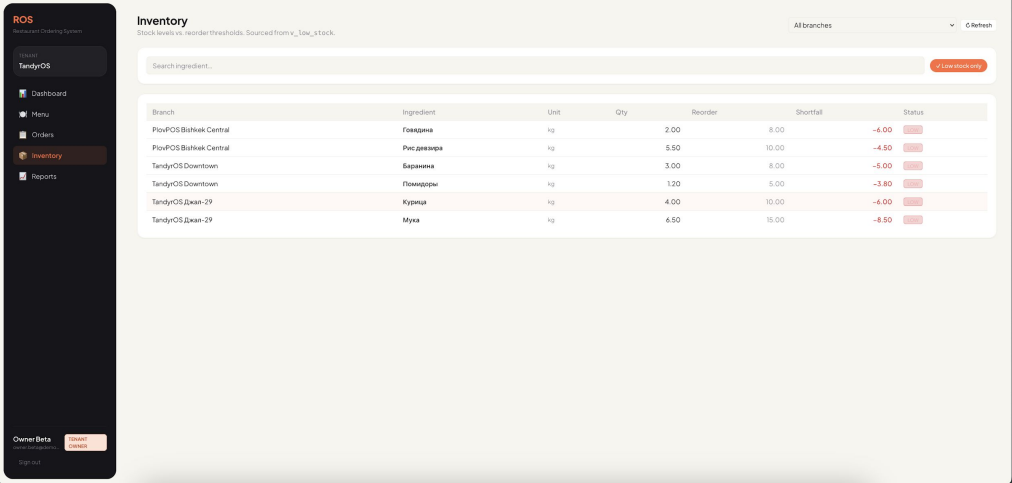
Calls `fn_place_order` on submit



Orders List

Partitioned orders table

Status flow: pending → completed



Inventory

`v_low_stock` view

CHECK (quantity >= 0)

Course Topics Applied

- ✓ 3rd Normal Form (3NF)
- ✓ Foreign Keys + ON DELETE policies
- ✓ CHECK constraints
- ✓ Indexes (B-Tree, GIN, BRIN)
- ✓ Table partitioning
- ✓ Row-Level Security
- ✓ Stored procedures (PL/pgSQL)
- ✓ Triggers (audit log)
- ✓ Materialized views
- ✓ ACID transactions

Thank you!